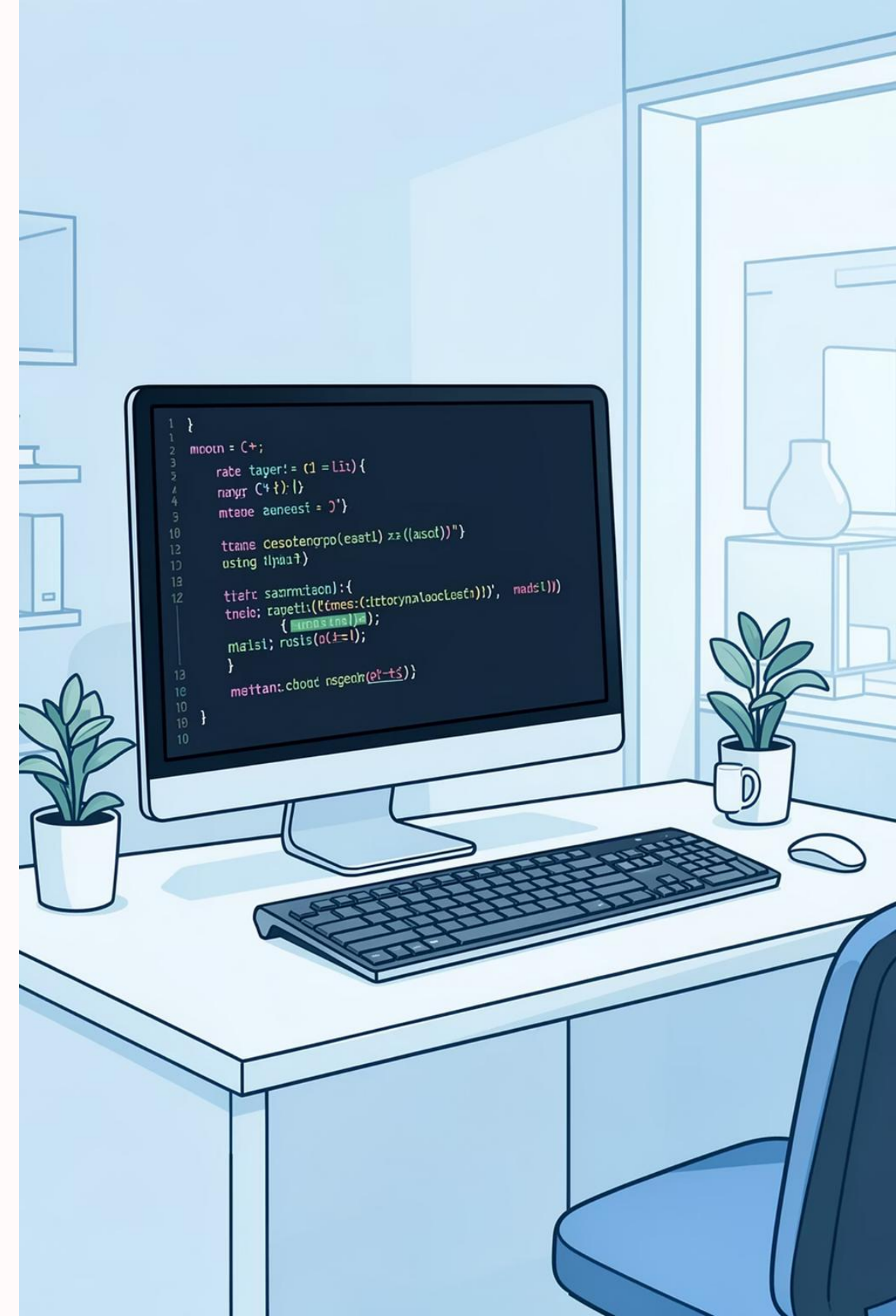


C++ Classes

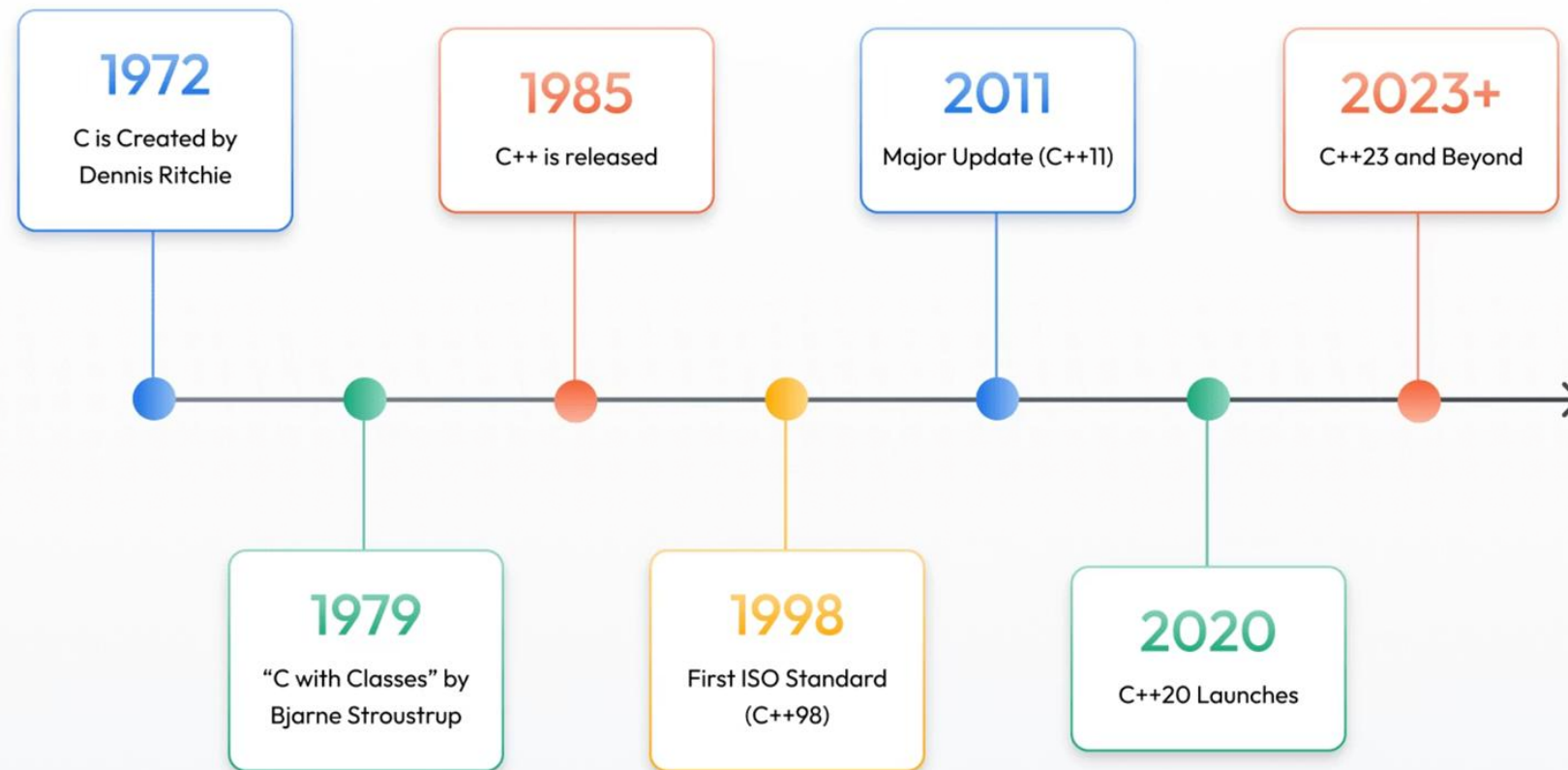
Introduction to C++ Programming

A comprehensive guide for mastering C++ from fundamentals to object-oriented concepts



A Brief History of

C++



Created by **Bjarne Stroustrup** in **1979** at Bell Labs

Originally called "**C with Classes**"

Evolved into **C++** in **1983**

Combined **efficiency** with **object-oriented** features

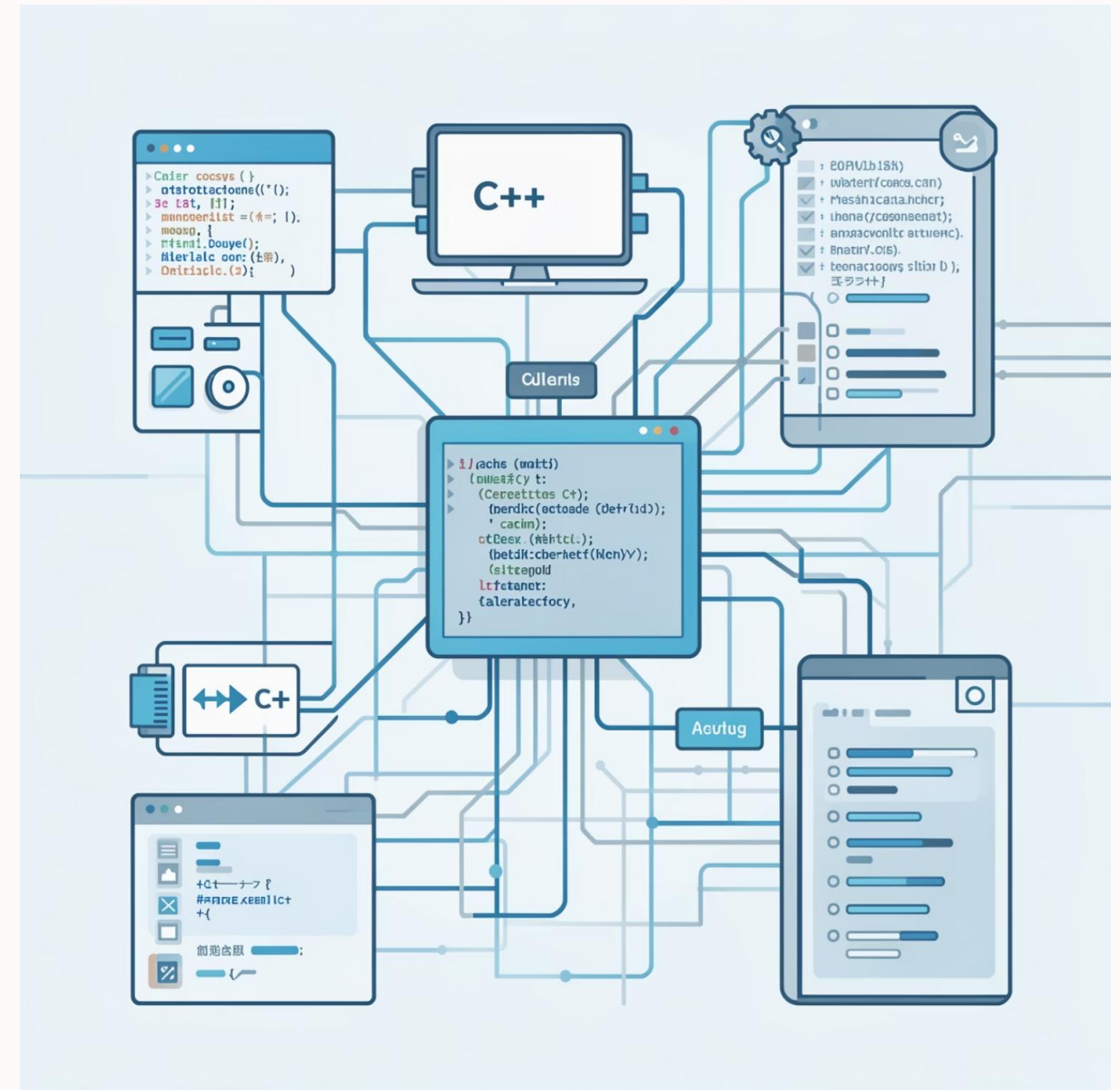
Why C++? Unpacking its Advantages

Provides **high performance** and system-level control, ideal for demanding applications.

Supports **object-oriented programming** (OOP), enabling modular and reusable code.

Powers a vast range of applications, including **games, operating systems, and embedded systems**.

Remains an **industry standard** for performance-critical software development.



Hello World

Example

Let's dive into your very first C++ program: the classic "Hello, World!" This simple program demonstrates the basic structure of a C++ application and how to print output to the console.

```
#include <iostream> // Required for input/output operations

int main() { // The main function where program execution begins
    std::cout << "Hello, World!" << std::endl; // Prints "Hello, World!" to the console
    return 0; // Indicates successful program termination
}
```

Understanding Hello World Code

Components

#include <iostream>: This line includes the **input/output stream library**, which is necessary for operations like **printing to the console**.

int main(): This is the **main() function**. Every C++ program must have a **main()** function, as it's the **entry point** where the program begins execution.

std::cout << "Hello, World!" << std::endl; This is the core of the program. **std::cout** is used to print data to the standard output (usually the console). The **<<** operator sends the string "Hello, World!" to **std::cout**. **std::endl** inserts a newline character and flushes the output buffer, ensuring the text appears immediately.

return 0; This statement indicates that the program has **terminated successfully**. A return value of **0** is conventionally used to signify success, meaning no errors occurred during execution.

Your First C++

Basic Syntax Rules

C++ is **case-sensitive**. This means ``myVariable`` and ``myvariable`` are treated as different identifiers.

Every statement in C++ must end with a **semicolon** (`;`), signaling the end of an instruction.

Curly braces (`{}`) are used to define blocks of code, such as function bodies or control flow statements.

You can add **comments** to your code using `//` for single-line comments or `/* ... */` for multi-line comments.

Before using a **variable**, it must be **declared** with its data type (e.g., ``int age;``, ``string name;``).

Understanding C++ Programme

Structure

1 Header Files (`#include`)

Preprocessor directives that include libraries. `<iostream>` provides input/output functionality.

0

3 Main Function (`int main`)

Programme entry point. Execution begins here. Must return an integer value to the operating system.

0

2 Namespaces (`using namespace std`)

Organises code elements. `std` contains standard library features like `cout` and `cin`.

0

4 Return Statement (`return 0`)

Indicates successful programme termination. Returns 0 to signal no errors occurred during execution.

📌 **Common Mistake:** Forgetting the semicolon after statements causes compilation errors. Missing `return 0;` generates warnings in some compilers.

Programme Execution Flow



From Code to Execution

C++ programmes follow a strict compilation process. The preprocessor handles directives, the compiler translates code to machine language, the linker combines object files, and finally the executable runs. **Key Point:** Compilation errors must be fixed before the programme can run. Runtime errors occur during execution.

Data Types in C++

In C++, **data types** are essential as they define the type of values a variable can hold and the operations that can be performed on them. Understanding data types is fundamental to writing effective and efficient C++ code.

Why Data Types

Matter

Memory Allocation: allocates specific **memory**

Range of Values: stores different **value ranges**

Operations: determines what **operations** can be done

Precision: affects decimal **accuracy**

Primitive Data Types in

1. int (Integers)

Stores **whole numbers**, positive or negative, typically 4 bytes.

```
int age = 30;  
int score = -100;
```

2. float (Single-Precision)

Stores **decimal numbers**, 4 bytes, with 6-7 decimal digits precision.

```
float pi = 3.14f;  
float price = 19.99f;
```

3. double (Double-Precision)

Stores **decimal numbers**, 8 bytes, with 15-17 decimal digits precision.

```
double gravity = 9.80665;  
double balance =  
12345.6789;
```

4. char (Characters)

Stores **single characters**, based on **ASCII values**, 1 byte.

```
char initial = 'J';  
char grade = 'A';
```

5. bool (Boolean)

Stores **true/false values**, 1 byte.

```
bool isActive = true;  
bool isFinished = false;
```

6. string (Text)

Stores **sequences of characters**, a standard library class.

```
string name = "Alice";  
string message = "Hello";
```

Data Type Size and Range

Reference

Understanding the size and range of C++ data types is crucial for efficient programming. **These properties dictate how much memory a variable consumes and the maximum and minimum values it can store.** Choosing the correct data type helps optimize resource usage and prevent unexpected data overflow or underflow issues.

Data Type Reference

Table

Data Type	Approximate Size (bytes)
int	4
float	4
double	8
char	1
bool	1
short	2
long	4 (or 8 on some systems)

Data Types: Code Examples

Here's a simplified C++ program demonstrating the **declaration, initialization**, and usage of basic data types.

```
#include <iostream> // For input/output operations

int main() { // Main function where program execution begins

    // Integer (int) example
    int age = 30;
    std::cout << "Age: " << age << std::endl; // Output: Age: 30

    // Floating-point (float) example
    float temperature = 25.5f;
    std::cout << "Temperature: " << temperature << std::endl; // Output: Temperature: 25.5

    // Double floating-point (double) example
    double price = 19.99;
    std::cout << "Price: " << price << std::endl; // Output: Price: 19.99

    // Character (char) example
    char grade = 'A';
    std::cout << "Grade: " << grade << std::endl; // Output: Grade: A

    // Boolean (bool) example
    bool is_valid = true;
    std::cout << "Is Valid: " << is_valid << std::endl; // Output: Is Valid: 1 (for true)

    return 0; // Indicates successful program execution
}
```

PROGRAM FOR SUM OF TWO NUMBERS

```
// Integer variable declaration and I/O example
#include <iostream>
using namespace std;

int main() {
    int num1, num2, sum;

    cout << "Enter two integer numbers: ";
    cin >> num1 >> num2;    // Input

    sum = num1 + num2;

    cout << "Sum = " << sum << endl;    // Output
    return 0;
}
```

Common Error: Using Uninitialized Variables

Code example showing the error:

```
int x;    // Uninitialized variable
cout << x;    // Using uninitialized variable
```

Actual Compiler Error/Warning:

"warning: 'x' is used **uninitialized** in this function [-Wuninitialized]" or "**error:** variable 'x' is **uninitialized** when used here"

Correct way:

```
int x = 0;    // Initialized variable
cout << x;    // Safe to use
```

To fix this, always **initialize** variables before their first use. Assigning a default value, like `0` for integers, ensures predictable program behavior.

Variables in C++: Declaration, Initialization, and Common Errors

In C++, a **variable is a named storage location that holds data**. It is a fundamental concept in programming, allowing you to store and manipulate information within your programs. Before you can use a variable, it must be **declared**.

Variable

Declaration

Variable declaration informs the compiler about the name and type of data a variable will hold. This reserves the appropriate amount of memory for the variable.

```
// Syntax: data_type variable_name;  
int age;           // Declares an integer variable named 'age'  
float price;       // Declares a floating-point variable named 'price'  
char initial;      // Declares a character variable named 'initial'
```

Variable

Initialization

Variable initialization is the process of assigning an initial value to a variable at the time of its declaration. It's good practice to always initialize variables to avoid unexpected behavior.

```
// Initialization examples  
int score = 100;      // Initializes 'score' with 100  
double pi = 3.14159;  // Initializes 'pi' with 3.14159  
bool isActive = true; // Initializes 'isActive' with true  
std::string name = "Alice"; // Initializes 'name' with "Alice"
```

Question 1: Variables and Data

Types

Write a program to declare variables of different data types and print their sizes using sizeof()

Answer 1: Variables and Data

Question:

Types

Write a program to input marks of 5 subjects and calculate total and average.

Code answer:

```
#include <iostream>
using namespace std;

int main() {
    int m1, m2, m3, m4, m5;
    float total, average;

    cout << "Enter 5 subject marks: ";
    cin >> m1 >> m2 >> m3 >> m4 >> m5;

    total = m1 + m2 + m3 + m4 + m5;
    average = total / 5;

    cout << "Total = " << total << endl;
    cout << "Average = " << average << endl;

    return 0;
```

INPUT:

```
Enter 5 subject marks:
80 75 90 85 70
```

OUTPUT:

```
Total = 400
Average = 80
```

Scope of Variables in

C++ the **scope** of a variable defines the region of the program where the variable can be accessed. Understanding **scope** is crucial for managing data and preventing naming conflicts in larger programs.

Local Variables

Local variables are declared inside a function or a block of code. They are only **accessible** within the function or block where they are defined and cease to exist once that function or block finishes execution.

```
#include <iostream>

void myFunction() {
    int localVariable = 10; // This is a local variable
    std::cout << "Inside myFunction, localVariable: " << localVariable << std::endl;
}

int main() {
    int mainLocalVariable = 20; // This is also a local variable
    std::cout << "Inside main, mainLocalVariable: " << mainLocalVariable << std::endl;
    myFunction();
    // std::cout << localVariable << std::endl; // ERROR: localVariable is not accessible here
    return 0;
}
```

Global Variables in

C++ In C++, **global variables** are declared outside of any function, making them **accessible** from any part of the program. Their **lifetime** extends throughout the **entire program execution**, meaning they are created when the program starts and destroyed when it ends. This allows different functions to share and modify the same data.

Understanding Global Variables

Here's an example demonstrating how a global variable is declared and used across multiple functions:

```
#include <iostream>

int globalVariable = 100; // Global variable

void display() {
    std::cout << globalVariable << std::endl;
}

int main() {
    std::cout << globalVariable << std::endl;
    display();
    return 0;
}
```

Block Scope in C++

Variables declared within curly braces {}, such as in if statements, for loops, or while loops, have **block scope**. This means they are created when the block is entered and destroyed when the block is exited. They are only **accessible** within that specific block.

```
// Example of block scope
#include <iostream>

int main() {
    int x = 10; // x has main function scope

    if (true) {
        int y = 20; // y has block scope within this if statement
        std::cout << "Inside if block: x = " << x << ", y = " << y << std::endl;
    }
    // std::cout << y << std::endl; // ERROR: y is not accessible here

    for (int i = 0; i < 2; ++i) {
        int z = 30; // z has block scope within this for loop
        std::cout << "Inside for loop: x = " << x << ", z = " << z << std::endl;
    }
    // std::cout << z << std::endl; // ERROR: z is not accessible here

    std::cout << "Outside blocks: x = " << x << std::endl;
    return 0;
}
```

Common Variable Errors in C++

1. Using Uninitialized Variables

Using an **uninitialized** variable means attempting to read its value before it has been assigned. This leads to unpredictable behavior or 'garbage' values.

```
int main() {  
    int x; // Declared but not initialized  
    std::cout << x << std::endl; // x contains garbage value  
    return 0;  
}
```

2. Declaring Variable

Twice

Attempting to declare a variable with the same name within the same scope more than once results in a **redeclaration** error, as the compiler requires unique names.

```
int main() {  
    int count = 10;  
    // int count = 20; // Error: redeclaration  
    std::cout << count << std::endl;  
    return 0;  
}
```

3. Type Mismatch

Type mismatch occurs when you try to **assign a value of one data type to a variable of an incompatible data type**. This can lead to data loss or incorrect behavior.

```
#include <iostream>

int main() {
    int myInt = 3.14; // Implicit conversion, data loss (myInt will be 3)
    std::cout << "myInt: " << myInt << std::endl;
    return 0;
}
```

4. Variable Name Conflicts

(Shadowing) Variable shadowing occurs when a **local variable** has the same name as a **variable in an outer scope**, such as a global variable. The local variable "shadows" or hides the outer variable within its own scope.

```
#include <iostream>

int x = 10; // Global variable

int main() {
    int x = 20; // Local variable 'x' shadows global 'x'
    std::cout << "Local x: " << x << std::endl; // Prints 20
    std::cout << "Global x (accessing via scope resolution): " << ::x << std::endl; // Prints 10
    return 0;
}
```

Input and Output in

C++ Input and Output (I/O) allows your program to communicate with users. **Input** receives data from the user, and **Output** displays data to the user.

What is Input

Input refers to the process of receiving data from an external source, typically from the user via the keyboard or from a file. This data is then processed by the program.

What is Output

Output refers to the process of displaying data or results from the program to an external destination, usually the console (screen) or a file. This allows users to see the program's actions and results.

Common I/O Operations

cout

Used to send data to the standard **output** stream (usually the console).

cin

Used to read **input** from the standard **input** stream (usually the keyboard).

endl

A manipulator that inserts a newline character into the **output** stream and flushes the buffer.

<< and >> Operators

The **<<** (insertion) operator is used with **cout** for **output**, and the **>>** (extraction) operator is used with **cin** for **input**.

Output Examples in C++

Output in C++ is done using cout (console output) to display data to the user.

Understanding Output with cout

Here are examples demonstrating various ways to use cout:

Simple output with cout

- Output multiple values
- Output with endl (newline)
- Output with variables
- Output with different data types

```
#include <iostream>
using namespace std;

int main() {
    // Simple output
    cout << "Hello, World!" << endl;

    // Output multiple values
    cout << "Name: " << "Alice" << endl;

    // Output with variables
    int age = 25;
    cout << "Age: " << age << endl;

    // Output different data types
    float price = 19.99f;
    cout << "Price: $" << price << endl;

    return 0;
}
```

Example Output

```
Hello, World!
Name: Alice
Age: 25
Price: $19.99
```


Input Examples in C++

Input in C++ is done using `cin` (console input) to receive data from the user.

Understanding Input with

Here are examples demonstrating various ways to use `cin`:

- Simple input from user
- Input multiple values
- Input with variables
- Input different data types
- Input with prompts

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Simple input
    int number;
    cout << "Enter a number: ";
    cin >> number;
    cout << "You entered: " << number << endl;

    // Input multiple values
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "Sum: " << (a + b) << endl;

    // Input string
    string name;
    cout << "Enter your name: ";
    cin >> name;
    cout << "Hello, " << name << endl;
```

Example Output

```
}
Enter a number: 42
You entered: 42
Enter two numbers: 10 20
Sum: 30
Enter your name: John
```

Arithmetic Operators in

Arithmetic operators perform mathematical operations on numbers.

C++

Addition +

adds two numbers

Subtraction -

subtracts two numbers

Multiplication *

multiplies two numbers

Division /

divides two numbers

Modulus %

returns remainder

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 3;

    cout << "a + b = " << (a + b) << endl; // 13
    cout << "a - b = " << (a - b) << endl; // 7
    cout << "a * b = " << (a * b) << endl; // 30
    cout << "a / b = " << (a / b) << endl; // 3
    cout << "a % b = " << (a % b) << endl; // 1

    return 0;
}
```

Output

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a % b = 1
```

Relational Operators in

C++

Relational operators compare two values and return true or false.

Greater than >

checks if left is greater

Less than <

checks if left is less

Equal to ==

checks if values are equal

Not equal to !=

checks if values are not equal

Greater than or equal >=

checks if left is greater or equal

Less than or equal <=

checks if left is less or equal

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;

    cout << "a > b: " << (a > b) << endl;    // 1 (true)
    cout << "a < b: " << (a < b) << endl;    // 0 (false)
    cout << "a == b: " << (a == b) << endl; // 0 (false)
    cout << "a != b: " << (a != b) << endl; // 1 (true)
    cout << "a >= b: " << (a >= b) << endl; // 1 (true)
    cout << "a <= b: " << (a <= b) << endl; // 0 (false)

    return 0;
}
```

Output

```
a > b: 1
a < b: 0
a == b: 0
a != b: 1
a >= b: 1
a <= b: 0
```

Logical Operators in C++

Logical operators combine multiple conditions and return true or false.

AND (&&)

returns true if both conditions are true

OR (||)

returns true if at least one condition is true

NOT (!)

reverses the boolean value

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5, c = 15;

    // AND operator
    cout << "(a > b && b < c): " << (a > b && b < c) << endl; // 1 (true)
    cout << "(a > b && b > c): " << (a > b && b > c) << endl; // 0 (false)

    // OR operator
    cout << "(a > c || b < c): " << (a > c || b < c) << endl; // 1 (true)
    cout << "(a > c || b > c): " << (a > c || b > c) << endl; // 0 (false)

    // NOT operator
    cout << "!(a > b): " << !(a > b) << endl; // 0 (false)
    cout << "!(a > c): " << !(a > c) << endl; // 1 (true)

    return 0;
```

Output

```
(a > b && b < c): 1
(a > b && b > c): 0
(a > c || b < c): 1
(a > c || b > c): 0
!(a > b): 0
!(a > c): 1
```

Assignment Operators in C++

Assignment operators assign values to variables.

Assignment =

assigns value to variable

Add and assign +=

adds and assigns

Subtract and assign -=

subtracts and assigns

Multiply and assign *=

multiplies and assigns

Divide and assign /=

divides and assigns

Modulus and assign %=

modulus and assigns

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;

    cout << "a = " << a << endl; // 10

    a += 5; // a = a + 5
    cout << "a += 5: " << a << endl; // 15

    a -= 3; // a = a - 3
    cout << "a -= 3: " << a << endl; // 12

    a *= 2; // a = a * 2
    cout << "a *= 2: " << a << endl; // 24

    a /= 4; // a = a / 4
    cout << "a /= 4: " << a << endl; // 6

    a %= 4; // a = a % 4
    cout << "a %= 4: " << a << endl; // 2
```

Output:

```
a = 10
a += 5: 15
a -= 3: 12
a *= 2: 24
a /= 4: 6
a %= 4: 2
```

Increment and Decrement Operators in C++

Increment and Decrement operators increase or decrease variable values by 1.

++ (increment) increases value by 1	-- (decrement) decreases value by 1	Pre-increment (++a) increments before using
Post-increment (a++) increments after using	Pre-decrement (--a) decrements before using	Post-decrement (a--) decrements after using

```
#include <iostream>
using namespace std;

int main() {
    int a = 5;

    cout << "a = " << a << endl; // 5

    cout << "++a (pre-increment): " << ++a << endl; // 6, then print
    cout << "a = " << a << endl; // 6

    cout << "a++ (post-increment): " << a++ << endl; // print 6, then increment
    cout << "a = " << a << endl; // 7

    cout << "--a (pre-decrement): " << --a << endl; // 6, then print
    cout << "a = " << a << endl; // 6

    cout << "a-- (post-decrement): " << a-- << endl; // print 6, then decrement
    cout << "a = " << a << endl; // 5
```

Output

```
a = 5
++a (pre-increment): 6
a = 6
a++ (post-increment): 6
a = 7
--a (pre-decrement): 6
a = 6
a-- (post-decrement): 6
a = 5
```

Ternary Operator in

The ternary operator is a shorthand for if-else statements. It takes three operands: condition ? value_if_true : value_if_false

C++

Syntax explanation

The basic syntax is condition ? expression_if_true : expression_if_false. The condition is evaluated first.

0

2

How it works

If the condition is true, expression_if_true is executed. Otherwise, expression_if_false is executed. The result of the chosen expression is then returned.

0

3

When to use

It's useful for concise conditional assignments or inline conditional logic, especially when dealing with simple if-else scenarios that return a value.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;
    int max;

    // Using ternary operator
    max = (a > b) ? a : b;
    cout << "Maximum: " << max << endl; // 10

    // Nested ternary
    int age = 20;
    string status = (age < 13) ? "Child" : (age < 18) ? "Teen" : "Adult";
    cout << "Status: " << status << endl; // Adult

    // In cout directly
    cout << "a is " << ((a > b) ? "greater" : "not greater") << " than b" << endl;

    return 0;
}
```

Output

```
Maximum: 10
Status: Adult
a is greater than b
```

Control Statements in

C++ control statements allow your program to **make decisions and execute different code blocks based on conditions**.

1

If Statement

An **if** statement executes a code block only if a **condition** is **true**.

```
#include <iostream>

int main() {
    int score = 95;
    if (score > 90) {
        std::cout << "Excellent!";
    }
    return 0;
}
```

2

If-Else Statement

An **if-else** statement executes one code block if a **condition** is **true**, and another if it's **false**.

```
#include <iostream>

int main() {
    int age = 20;
    if (age >= 18) {
        std::cout << "Adult" << std::endl;
    } else {
        std::cout << "Minor" << std::endl;
    }
    return 0;
}
```


Question 2: Control Statements

Write a program to check if a number is positive, negative, or zero using if-else

Answer 2: Control

Question:

Statements

Write a program to check if a number is positive, negative, or zero using if-else

Code answer:

```
#include <iostream>
using namespace std;

int main() {
    int num = 15;

    if (num > 0) {
        cout << num << " is positive" << endl;
    } else if (num < 0) {
        cout << num << " is negative" << endl;
    } else {
        cout << num << " is zero" << endl;
    }
}
```

```
return 0;
```

Output:

```
15 is positive
```

Switch Statement in

C++

A switch statement chooses one of many code blocks to execute based on a single expression.

Example: Switch

```
int day = 3;

switch (day) {
    case 1:
        std::cout << "Monday";
        break;
    case 2:
        std::cout << "Tuesday";
        break;
    default:
        std::cout << "Weekend";
        break;
}
```

Loops in C++

Loops allow you to execute a block of code multiple times. They are essential for repeating tasks efficiently.

1

For Loop

The **for loop** is used when you know in advance how many times you want to iterate. It consists of an initialization, a **condition**, and an **increment**/decrement statement.

```
for (int i = 0; i < 5; i++) {  
    // Code to be executed 5 times  
    std::cout << "Iteration: " << i << std::endl;  
}
```

2

While Loop

The **while loop** executes a block of code as long as a specified **condition** is true. The condition is evaluated before each **iteration**.

```
int i = 0;  
while (i < 3) {  
    // Code to be executed  
    std::cout << "Count: " << i << std::endl;  
    i++;  
}
```

Question 3: Loops

Write a program to print the multiplication table of a number using a for loop

Answer 3: Loops

Question:

Write a program to print the multiplication table of a number using a for loop

Code answer:

```
#include <iostream>
using namespace std;

int main() {
    int num = 5;

    cout << "Multiplication table of " << num << ":" << endl;
    for (int i = 1; i <= 10; i++) {
        cout << num << " x " << i << " = " << (num * i) << endl;
    }

    return 0;
}
```

Output:

```
Multiplication table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Arrays in C++

An **array** is a collection of **elements** of the same **data type** stored in contiguous **memory locations**.

Declaration, Initialization, and Access

```
// Declare and initialize an array of integers
int numbers[5] = {10, 20, 30, 40, 50};

// Accessing elements using their index (0-based)
std::cout << numbers[0]; // Output: 10 (first element)
std::cout << numbers[2]; // Output: 30 (third element)

// Common error: Index out of bounds
// For an array of size 5, valid indices are 0 to 4.
// Accessing numbers[5] would be an out-of-bounds error.
```

Array Length

```
// Get the number of elements in the array
int length = sizeof(numbers) / sizeof(numbers[0]);
std::cout << length; // Output: 5
```

Question 4: Arrays

Write a program to find the sum of all elements in an array

Answer 4: Arrays

Question:

Write a program to find the sum of all elements in an array

Code answer:

```
#include <iostream>
using namespace std;

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int size = 5;
    int sum = 0;

    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    cout << "Array elements: ";
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Sum of array elements: " << sum << endl;
```

Output:

```
Array elements: 10 20 30 40 50
Sum of array elements: 150
```

Strings in C++

A **string** is a sequence of **characters** used to represent text. In C++, strings are objects from the standard library.

Declaration and Initialization

```
#include <string> // For std::string
#include <iostream>

int main() {
    std::string greeting = "Hello, C++!";
    std::string name;
    name = "World";
    std::cout << greeting << " " << name << std::endl;
    return 0;
}
```

Common String Operations

1

Getting Length

The **length** of a string is the number of characters it contains.

(.length())

```
std::string s = "ANIMAL";
int len = s.length(); // len will be 5
std::cout << len << std::endl;
```

2

Accessing

Individual **characters** in a string can be accessed using their 0-based index.

Characters

```
std::string s = "Hello";
char first = s[0]; // 'H'
char last = s[4];  // 'o'
std::cout << first << last << std::endl;
```

Functions in C++

A **function** is a **reusable block of code that performs a specific task**. Functions promote code reuse and modularity.

Declaration and Definition

A function must be declared before it can be used. The declaration (prototype) tells the compiler about the **function's** name, **return** type, and **parameters**. The definition provides the actual code of the **function**.

```
// Function Declaration (prototype)
```

```
int add(int a, int b);
```

```
// Function Definition
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

Common Operations

Function with

Parameters

Functions can accept input values, called **parameters**, to perform their tasks.

```
void greet(std::string name) {  
    std::cout << "Hello, " << name << "!" << std::endl;  
}
```

Function with Return

Value

Functions can compute a result and send it back to the **calling** code using a **return** statement.

```
int multiply(int x, int y) {  
    return x * y;  
}
```

Function Call & Passing

Arguments

To execute a **function**, you **call** it by its name, providing actual values (known as **arguments**) for its **parameters**.

```
int main() {  
    greet("Alice"); // Calling greet, "Alice" is the argument  
    int product = multiply(5, 3); // Calling multiply, 5 and 3 are arguments  
    std::cout << product << std::endl; // Output: 15  
    return 0;  
}
```

Question 5:

Functions

Write a function that takes two numbers as parameters and returns their sum

Answer 5:

Question:

Functions

Write a function that takes two numbers as parameters and returns their sum

Code answer:

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int main() {
    int num1 = 25;
    int num2 = 35;
    int result = add(num1, num2);

    cout << "Number 1: " << num1 << endl;
    cout << "Number 2: " << num2 << endl;
    cout << "Sum: " << result << endl;

    return 0;
}
```

Output:

```
Number 1: 25
Number 2: 35
Sum: 60
```

OOPS

FUNDAMENTALS

Understanding Object-Oriented Programming (OOP)

C++ is fundamentally an object-oriented language. OOP is a programming paradigm that **organises software design around data, or objects, rather than functions and logic**. It's about modelling the real world into code, making programs more intuitive, maintainable, and reusable.



Modularity

Break complex problems into smaller, manageable objects, making code easier to develop and debug.



Security

Control access to data within objects, protecting it from unintended external modification.



Reusability

Create generic templates (classes) that can be reused to build new objects without writing the same code twice.



Maintainability

Changes in one part of the code are less likely to affect other parts, simplifying updates and fixes.

Classes and Objects in C++

A **class** is a blueprint for creating **objects**. An **object** is an instance of a **class** that contains data (**attributes**) and functions (**methods**).

What is a Class?

In C++, a **class** is a user-defined data type or a blueprint from which **objects** are created. It defines a set of properties (**attributes**) and behaviors (**methods**) that all **objects** of that **class** will have. Think of a **class** like the architectural plans for a house: it details what the house will look like, its components, and its functionality, but it is not the house itself.

What is an Object?

An **object** is a concrete instance of a **class**. While the **class** is the blueprint, the **object** is the actual entity created from that blueprint. Each **object** created from a **class** occupies memory and has its own unique set of **attributes** (data) based on the **class** definition. Using the house analogy, an **object** is the actual house built according to the architectural plans defined by the **class**.

Simple Code Example

Here's a basic C++ example demonstrating how to define a **class**, create an **object**, and access its **attributes** and **methods**.

```
class Student {
public:
    string name; // Attribute (data member)
    int age;      // Attribute (data member)

    // Method (member function)
    void display() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    Student s1;          // Create an object 's1' of the Student class
    s1.name = "John";    // Access and set attribute 'name'
    s1.age = 20;         // Access and set attribute 'age'
    s1.display();        // Call method 'display' for object 's1'

    Student s2;          // Create another object 's2'
    s2.name = "Alice";
    s2.age = 22;
    s2.display();
    return 0;
}
```

Constructors and Destructors in

C++ Constructors initialize objects when created. **Destructors** clean up resources when objects are destroyed.

Constructors

A **constructor** is a special member function that is automatically called when an object of a class is created. It is used to initialize the object's member variables.

```
class Student {
public:
    string name;
    int age;
    Student(string n, int a) { // Constructor with parameters
        name = n;
        age = a;
    }
};

// Usage: Student s("John", 20);
```

Destructors

A **destructor** is a special member function that is automatically called when an object is destroyed. It is used to perform any necessary cleanup, such as deallocating memory.

```
class FileHandler {
public:
    FileHandler() { // Constructor
        cout << "File opened";
    }
    ~FileHandler() { // Destructor
        cout << "File closed";
    }
};
```

Access Specifiers in C++

Access specifiers control the visibility of class members. They determine what parts of a class can be accessed from outside the class.

PUBLIC

Members declared as **public** are **accessible** from outside the class.

```
class BankAccount {
public:
    void deposit(int amount) { /* ... */ }
};
```

PRIVATE

Members declared as **private** are only **accessible** from within the same class. This is key for **encapsulation**.

```
class BankAccount {
private:
    int balance; // Cannot access directly from outside
public:
    void deposit(int amount) { balance += amount; }
};
```

PROTECTED

Members declared as **protected** are **accessible** within the same class and by derived classes (inheritance).

```
class Vehicle {
protected:
    string color; // Accessible by derived classes
};
class Car : public Vehicle {
public:
    void setColor(string c) { color = c; }
};
```

OOPS PRINCIPLES

Encapsulation in C++

Encapsulation is bundling data (attributes) and methods (functions) together within a class, and hiding internal details from the outside world using access specifiers.

It's a core OOP concept that helps to keep data safe from external, unauthorized access and modification. By making certain members **private**, you restrict direct access, forcing interaction through **public** methods. This mechanism ensures that the object's internal state remains consistent and valid.

Real-world Example: BankAccount

```
class BankAccount {  
private:  
    double balance; // Private data (data hiding)  
  
public:  
    BankAccount(double initialBalance = 0) : balance(initialBalance) {}  
  
    void deposit(double amount) {  
        if (amount > 0) { // Validation logic  
            balance += amount;  
        }  
    }  
  
    void withdraw(double amount) {  
        if (amount > 0 && balance >= amount) { // Validation logic  
            balance -= amount;  
        }  
    }  
  
    double getBalance() const { // Public method to access data  
        return balance;  
    }  
};
```

Abstraction in C++

Abstraction is the process of hiding complex implementation details and showing only the essential features of an object. It allows users to interact with objects without knowing how they work internally.

Abstraction focuses on what an object does rather than how it does it. It helps in managing complexity by presenting a high-level view of functionality, making systems easier to understand and maintain.

Real-world Example: Shape Class

```
class Shape { // Abstract base class
public:
    // Pure virtual function
    virtual double calculateArea() = 0;
    virtual ~Shape() {} // Virtual destructor
};

class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    double calculateArea() override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
private:
    double length;
    double width;
public:
    Rectangle(double l, double w) : length(l), width(w) {}
    double calculateArea() override {
        return length * width;
    }
};
```

Inheritance in C++

Inheritance is a mechanism that allows a class (**derived class**) to inherit properties and methods from another class (**base class**). It promotes code reuse and establishes a hierarchical relationship between classes.

Real-world Example: Vehicle

```
// C++ Code Example: Vehicle Inheritance

class Vehicle { // Base class
public:
    string brand = "Ford";
    void honk() {
        cout << "Tuut, tuut!" ;
    }
};

class Car : public Vehicle { // Derived class
public:
    string model = "Mustang";
};
```

```
// Usage: Car myCar; myCar.honk();
```

Benefits of Inheritance

Code Reuse

Avoids redundant code by allowing derived classes to utilize base class implementations.

Hierarchy

Establishes a clear parent-child relationship between classes, aiding in organization.

Extensibility

Easily extend existing functionality by creating new derived classes without modifying base classes.

Polymorphism in C++

Polymorphism means 'many forms'. It allows objects of different classes to be treated as objects of a common **base class**. It enables a single function or method to work with different types of data. Polymorphism is a fundamental concept in Object-Oriented Programming (OOP) that enables code flexibility and extensibility. It allows you to define one interface and have multiple implementations.

Real-world Example: Animal Class

```
class Animal { // Base class
public:
    virtual void speak() { // Virtual method
        cout << "Animal makes a sound" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() override { // Overriding base class method
        cout << "Woof!" << endl;
    }
};

class Cat : public Animal {
public:
    void speak() override {
        cout << "Meow!" << endl;
    }
};

class Bird : public Animal {
public:
    void speak() override {
        cout << "Chirp!" << endl;
    }
};

// Usage:
// Animal* myAnimal;
// myAnimal = new Dog();
// myAnimal->speak(); // Output: Woof!
```

Benefits of Polymorphism

Flexibility Allows objects of different types to be processed using a single interface, making code more adaptable.	Extensibility New derived classes can be added without modifying existing code that uses the base class interface.	Code Simplicity Reduces the need for complex conditional statements (if-else or switch) when dealing with different object types.
---	---	---

THANK YOU